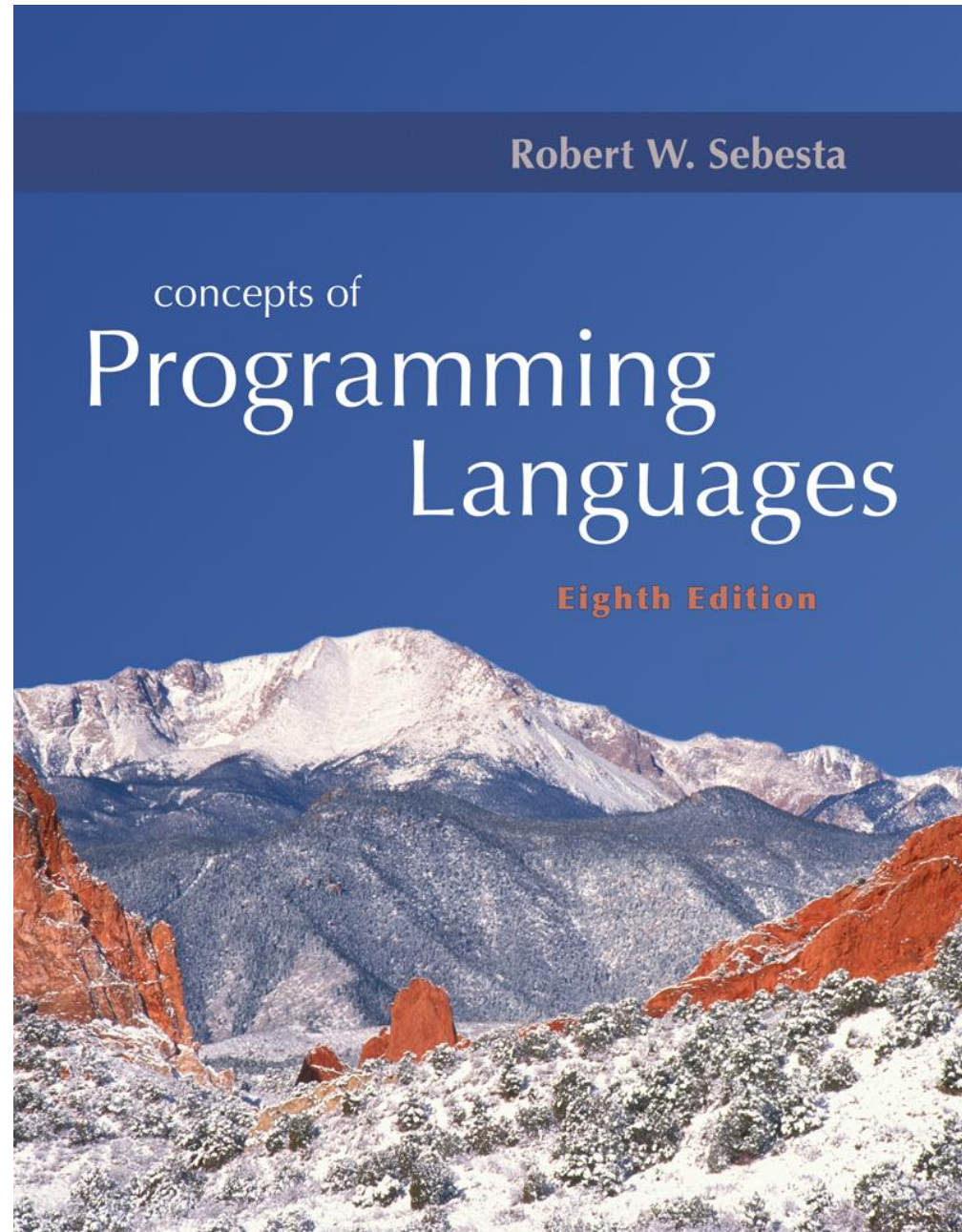


Chapter 5

Names, Bindings,
Type Checking, and
Scopes



ISBN 0-321-49362-1

Compile Online

- <http://codepad.org/>
- <http://compileonline.com/>
- <https://www.jdoodle.com>
- <https://ideone.com/>
- <http://coliru.stacked-crooked.com/>
- <https://www.onlinegdb.com/>
- <https://repl.it/>
- <https://www.python.org/shell/>
- <http://rextester.com/>
- <http://www.compileonline.com/>

Chapter 5 Topics

- Introduction
- Names
- Variables
- The Concept of Binding
- Type Checking
- Strong Typing
- Type Compatibility
- Scope and Lifetime
- Referencing Environments
- Named Constants

Introduction

- Imperative languages are abstractions of von Neumann architecture
 - Memory
 - Processor
- Variables characterized by attributes
 - Type: to design, must consider scope, lifetime, type checking, initialization, and type compatibility

Names

- Design issues for names:
 - Maximum length?
 - Are connector characters allowed?
 - Are names case sensitive?
 - Are special words reserved words or keywords?

Names (continued)

- Length
 - If too short, they cannot be connotative
 - Language examples:
 - FORTRAN I: maximum 6
 - COBOL: maximum 30
 - FORTRAN 90 and ANSI C: maximum 31
 - Ada and Java: no limit, and all are significant
 - C++: no limit

Names (continued)

- Case sensitivity
 - Disadvantage: readability (names that look alike are different)
 - worse in C++ and Java because predefined names are mixed case (e.g. `IndexOutOfBoundsException`)
 - C, C++, and Java names are case sensitive
 - The names in other languages are not

Names (continued)

- Special words
 - An aid to readability; used to delimit or separate statement clauses
 - A *keyword* is a word that is special only in certain contexts, e.g., in Fortran
 - `Real VarName` (*Real is a data type followed with a name, therefore Real is a keyword*)
 - `Real = 3.4` (*Real is a variable*)
 - A *reserved word* is a special word that cannot be used as a user-defined name

Variables

- A variable is an abstraction of a memory cell
- Variables can be characterized as a sextuple of attributes:
 - Name
 - Address
 - Value
 - Type
 - Lifetime
 - Scope

Variables Attributes

- Name – not all variables have them
- Address – the memory address with which it is associated
 - A variable may have different addresses at different times during execution
 - A variable may have different addresses at different places in a program
 - If two variable names can be used to access the same memory location, they are called **aliases**
 - Aliases are created via pointers, reference variables, C and C++ unions
 - Aliases are harmful to readability (program readers must remember all of them)

Variables Attributes (continued)

- *Type* – determines the range of values of variables and the set of operations that are defined for values of that type; in the case of floating point, type also determines the precision
- *Value* – the contents of the location with which the variable is associated
- *Abstract memory cell* – the physical cell or collection of cells associated with a variable

The Concept of Binding

- The l-value of a variable is its address
- The r-value of a variable is its value
- A *binding* is an association, such as between an attribute and an entity, or between an operation and a symbol
- *Binding time* is the time at which a binding takes place.

Possible Binding Times

- Language design time -- bind operator symbols to operations
- Language implementation time-- bind floating point type to a representation
- Compile time -- bind a variable to a type in C or Java
- Load time -- bind a FORTRAN 77 variable to a memory cell (or a C `static` variable)
- Runtime -- bind a nonstatic local variable to a memory cell

Static and Dynamic Binding

- A binding is *static* if it first occurs before run time and remains unchanged throughout program execution.
- A binding is *dynamic* if it first occurs during execution or can change during execution of the program

Type Binding

- How is a type specified?
- When does the binding take place?
- If static, the type may be specified by either an explicit or an implicit declaration

Explicit/Implicit Declaration

- An *explicit declaration* is a program statement used for declaring the types of variables
- An *implicit declaration* is a default mechanism for specifying types of variables (the first appearance of the variable in the program)
- FORTRAN, BASIC, and Perl provide implicit declarations
 - Advantage: writability
 - Disadvantage: reliability (less trouble with Perl)

Perl Implicit Declaration Example

- Perl does not require separate declaration of variables; they are defined implicitly within expressions

```
$a = 4;  
$b = "a string of words";  
$c = 4.5; # a decimal number  
$d = 3.14e10; # a floating point number
```

- Use **\$** prefix for *scalar*
- **@** prefix for *array* and **%** for *associative array*

Basic Qbasic (QB64) Implicit Declaration Example

- Basic does not require separate declaration of variables

```
a = 5  
age% = 13  
s$5 = "Hi" ' a string
```

- Use \$ suffix for *string*
- % suffix for *integer* and ~% for *unsigned integer*

Dynamic Type Binding

- Dynamic Type Binding (JavaScript and PHP)
- Specified through an assignment statement
e.g., JavaScript

```
list = [2, 4.33, 6, 8];
```

```
list = 17.3;
```

- Advantage: flexibility (generic program units)
- Disadvantages:
 - High cost (dynamic type checking and interpretation)
 - Type error detection by the compiler is difficult

Categories of Variables by Lifetimes

- **Static**--bound to memory cells before execution begins and remains bound to the same memory cell throughout execution, e.g., all FORTRAN 77 variables, C static variables
 - **Advantages:** efficiency (direct addressing), history-sensitive subprogram support
 - **Disadvantage:** lack of flexibility (no recursion)

C Code, Non-static Local Variable

```
#include <stdio.h>

void fun1 (){
    int x = 7;
    printf("%d\n",x++);
}

int main ()
{
    fun1();
    fun1();
    fun1();
}
```

C Code, Static Local Variable

```
#include <stdio.h>

void fun1 (){
    static int x = 7;
    printf("%d\n",x++);
}

int main ()
{
    fun1();
    fun1();
    fun1();
}
```

Categories of Variables by Lifetimes

- Stack–dynamic--Storage bindings are created for variables when their declaration statements are elaborated.
- If scalar, all attributes except address are statically bound
 - local variables in C subprograms and Java methods
- Advantage: allows recursion; conserves storage
- Disadvantages:
 - Overhead of allocation and deallocation
 - Subprograms cannot be history sensitive
 - Inefficient references (indirect addressing)

Categories of Variables by Lifetimes

- *Explicit heap-dynamic* -- Allocated and deallocated by explicit directives, specified by the programmer, which take effect during execution
- Referenced only through pointers or references, e.g. dynamic objects in C++ (via new and delete), all objects in Java
- Advantage: provides for dynamic storage management
- Disadvantage: inefficient and unreliable

Categories of Variables by Lifetimes

- *Implicit heap-dynamic*--Allocation and deallocation caused by assignment statements
 - all variables in APL; all strings and arrays in Perl and JavaScript
- Advantage: flexibility
- Disadvantages:
 - Inefficient, because all attributes are dynamic
 - Loss of error detection

Type Checking

- Generalize the concept of operands and operators to include subprograms and assignments
- *Type checking* is the activity of ensuring that the operands of an operator are of compatible types
- A *compatible type* is one that is either legal for the operator, or is allowed under language rules to be implicitly converted, by compiler-generated code, to a legal type
 - This automatic conversion is called a coercion.
- A *type error* is the application of an operator to an operand of an inappropriate type

Type Checking (continued)

- If all type bindings are static, nearly all type checking can be static
- If type bindings are dynamic, type checking must be dynamic
- A programming language is *strongly typed* if type errors are always detected

Strong Typing

- ***Strong typing*** facility is used to prevent mixing of different types

```
int x=5;  
float y = 3.1*x;
```

- Advantage of strong typing: allows the detection of the misuses of variables that result in type errors
- Some programming languages make it easy to use a value of one type as if it were a value of another type. This is sometimes described as "weak typing"
- It is affected by:
 - Aliasing
 - Coercion
 - Type casting
 - Union (not all)

Strong Typing (Cont')

- Language examples:
 - FORTRAN 77 is not strong typing: EQUIVALENCE
 - EQUIVALENCE STATEMENT is a specification statement which causes two or more items (variables or arrays) to be associated with each other, i.e. to correspond to the same area of memory
 - Provide a simple variable name as an alias
 - Complex item which is associated with two real ones
 - Map the two-dimensional array onto the one-dimensional array
 - Character items can only be associated with other character items

*CHARACTER S*80, A(80)*1*
EQUIVALENCE (S, A)

Strong Typing (Cont')

- Language examples:
 - Pascal is not strong typing: variant type
 - Assign any simple type of values in a variant variable. The type of a value stored in a variant is only determined at runtime

```
Program UsingVariant;  
uses variants;  
var  
  v : variant;  
  i : integer;  
  r: real;  
  as : ansistring;  
  
begin  
  i := 100;  
  v:= i;  
  writeln('Variant as Integer: ', v);  
  r:= 234.345;  
  v:= r;  
  writeln('Variant as real: ', v);  
  as:= ' bla bla bla';  
  v:= as;  
  writeln('Variant as AnsiString: ', v);  
end.
```

Strong Typing (Cont')

- Language examples:
 - C and C++ are not: parameter type checking can be avoided; unions are not type checked
 - A **union** is a special data type available in C that allows to store different data types in the same memory location
 - A union may have many members, but only one member can contain a value at any given time
 - Unions provide an efficient way of using the same memory location for multiple-purpose

Strong Typing (Cont')

- Language examples:

```
#include <stdio.h>
union Data{
    int y;
    int x;
};
int main() {
    union Data data;
    data.y = 1;
    //data.x = 1;
    printf( "data.y : %d\n", data.y);
    printf( "data.x : %d\n", data.x);
    return 0;
}
```

data.y : 1
data.x : 1

Strong Typing (Cont')

- Language examples:
 - Ada is, almost (UNCHECKED CONVERSION is loophole)
 - An unchecked conversion is a **bit-for-bit copy** without regard to the meanings attached to those bits and bit positions
 - Ensure that the value resulting from *Ada.Unchecked_Conversion* properly represents a value of the parameter's **subtype**
 - Using the '*Valid*' attribute on scalar data allows you to check whether it is in range without raising an exception if it is out of range.

Name Type Compatibility

- *Name type compatibility* means the two variables have compatible types if they are in either the same declaration or in declarations that use the same type name
- Easy to implement but highly restrictive:
 - Subranges of integer types are not compatible with integer types
 - Formal parameters must be the same type as their corresponding actual parameters (Pascal)

Structure Type Compatibility

- *Structure type compatibility* means that two variables have compatible types if their types have identical structures
- More flexible, but harder to implement

Type Compatibility (continued)

- Consider the problem of two structured types:
 - Are two record types compatible if they are structurally the same but use different field names?
 - Are two array types compatible if they are the same except that the subscripts are different? (e.g. [1..10] and [0..9])
 - Are two enumeration types compatible if their components are spelled differently?
 - With structural type compatibility, you cannot differentiate between types of the same structure (e.g. different units of speed, both float)

Variable Attributes: Scope

- The *scope* of a variable is the range of statements over which it is visible
- The *nonlocal variables (globals)* of a program unit are those that are visible but not declared there
- The scope rules of a language determine how references to names are associated with variables

Static Scope

- Based on program text
- To connect a name reference to a variable, you (or the compiler) must find the declaration
- Search process: search declarations, first locally, then in increasingly larger enclosing scopes, until one is found for the given name
- Enclosing static scopes (to a specific scope) are called its static ancestors; the nearest static ancestor is called a static parent

Scope (continued)

- Variables can be hidden from a unit by having a "closer" variable with the same name
- C++ and Ada allow access to these "hidden" variables
 - In Ada: `unit.name`
 - In C++: `class_name::name`

Blocks

- Allowing a section of code to have its own local variables
- Such variables are typically stack dynamic, so their storage is allocated when the section is entered and deallocated when the section is exited
- The C-based languages allow any compound statement (a statement sequence surrounded by matched braces) to have declarations and thereby define a new scope

C Language (Local Scope)

```
#include<stdio.h>  
int x = 40 ; // Scope : Whole Program (Global)  
int main() {  
    int x = 10 ; // Scope : In main  
    {  
        int x = 30 ; // Scope : within a block  
        for(int i =0;i<5;i++){  
            int x = 30 ; // Scope : within for block  
        }  
        printf("%d",x);  
    }  
    printf("%d",x);  
}
```

Nested Subroutines

- Nested subroutines usually inherit the variable scope of the "parent" function
- Parameters are generally still possible where needed or helpful in sub-functions
- Examples of nested functions: Pascal, Ada and Php
- Reusable in another context

Pascal (Nested Subroutines)

```
function E(x: real): real;  
    function F(y: real): real;  
    begin  
        F := x + y  
    end;  
begin  
    E := F(3) + F(4)  
end;
```

- <https://ideone.com/sg9TYE>

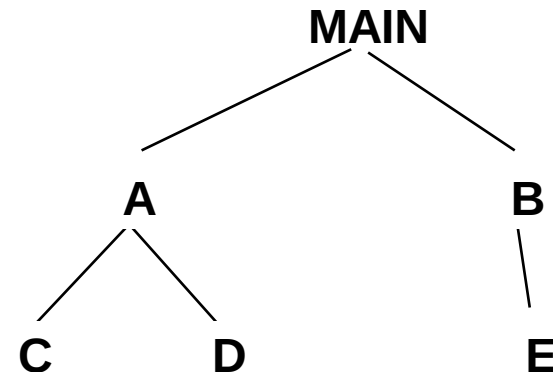
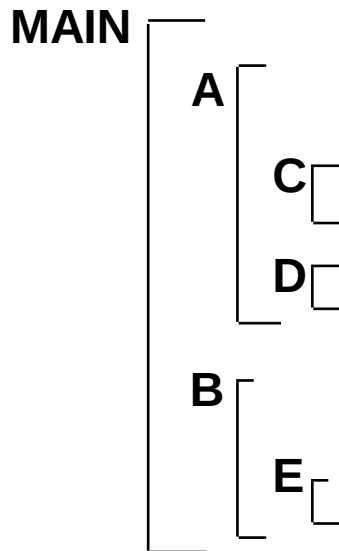
Lifetime and Scope

- Both lifetime and scope should exist to access a variable

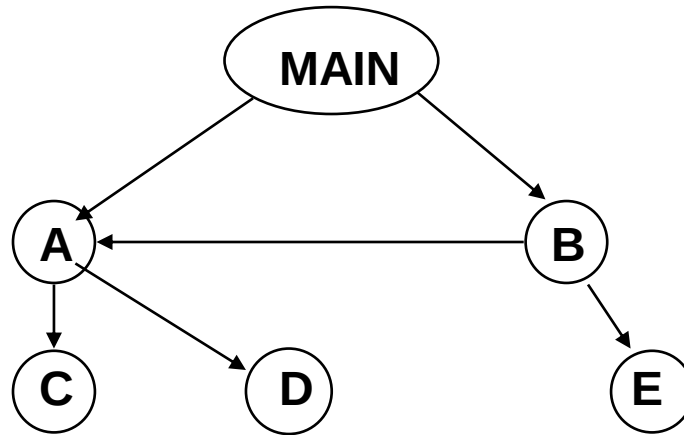
Lifetime without Scope (C Language)	Scope without Lifetime (Java)
<pre>#include <stdio.h> void fun1 (){ static int x = 0; } int main () { fun1 (); printf("%d\n", x); }</pre>	<pre>Book b1;</pre>

Evaluation of Static Scoping

- Assume MAIN calls A and B
A calls C and D
B calls A and E



Static Scope Example



Static Scope (continued)

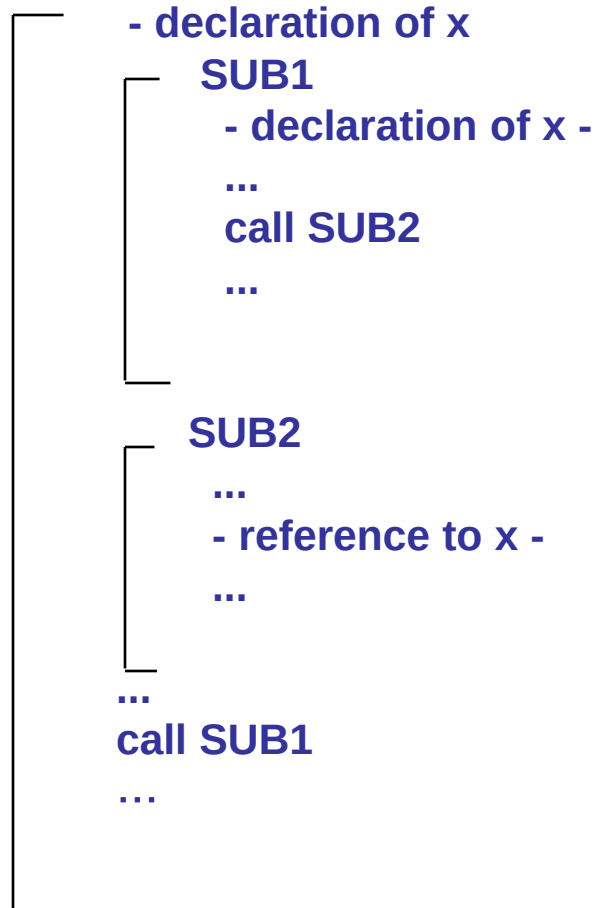
- Suppose the spec is changed so that D must now access some data in B
- Solutions:
 - Put D in B (but then C can no longer call it and D cannot access A's variables)
 - Move the data from B that D needs to MAIN (but then all procedures can access them)
- Overall: static scoping often encourages many globals

Dynamic Scope

- Based on calling sequences of program units, not their textual layout (temporal versus spatial)
- References to variables are connected to declarations by searching back through the chain of subprogram calls that forced execution to this point

Scope Example

MAIN



**MAIN calls SUB1
SUB1 calls SUB2
SUB2 uses x**

Scope Example

- Static scoping
 - Reference to x is to MAIN's x
- Dynamic scoping
 - Reference to x is to SUB1's x
- Evaluation of Dynamic Scoping:
 - Advantage: convenience
 - Disadvantage: poor readability

Referencing Environments

- The *referencing environment* of a statement is the collection of all names that are visible in the statement
- In a static-scoped language, it is the local variables plus all of the visible variables in all of the enclosing scopes
- In a dynamic-scoped language, the referencing environment is the local variables plus all visible variables in all active subprograms
- A subprogram is **active** if its execution has begun but has not yet terminated

Scope and Lifetime

- Scope and lifetime are sometimes closely related, but are **different concepts**
- Consider a `static` variable in a C or C++ function

Example: Consider the following C function:

```
void fun() {  
    int a, b, c;      /* Definition No.1 */  
    ...  
    while (.....) {  
        int b, c, d;   /* Definition No.2 */  
        ... <----- 1  
        if (.....) {  
            int c, d, e; /* Definition No.3 */  
            ... <----- 2  
        }  
        ... <----- 3  
    }  
    ... <----- 4  
  
} //end fun
```

Example (Cont')

- For each of the four marked points in this function, list each visible variable along with the number of the definition statement that defines it

Points	Variables with Definition Statement Number
1	<i>b, c and d of Def. No.2</i> <i>a of Def No. 1</i>
2	<i>c, d and e of Def. No. 3</i> <i>b of Def. No.2</i> <i>a of Def No. 1</i>
3	
4	

Summary

- Case sensitivity and the relationship of names to special words represent design issues of names
- Variables are characterized by the sextuples: name, address, value, type, lifetime, scope
- Binding is the association of attributes with program entities
- Scalar variables are categorized as: static, stack dynamic, explicit heap dynamic, implicit heap dynamic
- Strong typing means detecting all type errors